



Algorithms and Data Structures in C

Practical III - "Spell Checker"

Deadline: Monday, March 2, 2026, at 9am

Introduction

In this assignment, you will develop a spell checker. There are two parts: First you will make a basic spell checker, then you will improve its performance using a trie. Both parts should be submitted to Themis before the same deadline.

Input-output behavior

For both parts of this assignment, we use the same input and output format. The input consists of the dictionary and the text to be checked, after each other.

- The dictionary consists of one word per line. Words consist of letters from 'a' to 'z', in lower or upper case. You can assume that no word is longer than 45 characters.
- You are dealing with a very strange dictionary: It is not alphabetically sorted.
- The dictionary and the text to be checked are separated by a single line with '!'.
- The text to be checked consists of letters from 'a' to 'z' in lower and upper case, digits from '0' to '1', spaces, newline characters, and the following symbols: . , ' / - " : ; ? ! ().
- Digits, spaces, newline characters, and any of the above symbols denote word boundaries. Your program should output all words that occur in the text but not in the dictionary (one per line), followed by the number of unknown words. Your program should ignore lower/upper case differences in the input and print the unknown words in lowercase.

Example input and output:

Input:

```
one
two
three
and
four
!
hello?
one,two; three!
And - "for".
```

Output:

```
hello
two
for
3
```

Part 1: the basic spell checker

1. Download `spellchecker.zip` from Themis and unpack it.
2. The file `speller.c` contains a basic spellchecker. The code compiles, but the resulting program ignores some of the rules above. Your first task is to repair the program.
3. The file `exampleInput.txt` contains the example above. The `Makefile` includes some useful commands. Use `'make example'` to run your program and `'make check-example'` to compare the output of your program against `'exampleOutput.txt'`. You can also use `'make debug-example'` to run your program with Valgrind.
4. For this part you only have to upload `speller.c` when submitting to Themis. The files `dictionary.h`, `dictionary.c`, `memory.h` and `memory.c` are included automatically.

Part 2: the improved version

1. After you have a solution for Part 1, run `'make time-largeExample'`. As you can see in the `Makefile` this will run `time ./speller < large_exampleInput.txt`. Probably your program will take quite long for this example with a real English dictionary.
2. Your new goal is thus to make your spell checker faster, *using a trie*. You should now modify `dictionary.h` and `dictionary.c`. Ideally you should not (have to) change anything in `speller.c`.
3. For this part, only `memory.h` and `memory.c` are included automatically by Themis, so you should upload `speller.c`, `dictionary.c` and `dictionary.h`. In fact, you can upload your solution from Part 1 and the unmodified `dictionary.c` and `dictionary.h` to Part 2, and the program will compile and run, but it will not be fast enough to pass the tests.



Note.

You may use this definition for a standard trie:

```
#include <stdbool.h>

typedef struct TrieNode* Trie;

struct TrieNode {
    bool endNode;
    Trie children[26];
};
```



Report

Additionally, you will write a report on this assignment as well. You can find the details about what these reports should look like on Brightspace (more specifically: Appendix E of the *Lecture notes* and the *Grading Document*). Note that writing a report for this assignment is **mandatory!** Not submitting a report will result in your code submissions not being considered.

You are required to use L^AT_EX for the report, using the provided template on Brightspace.

Note: Please do not include long dictionaries in your report PDF.

Grading

Your grade is determined as follows:

- The basic implementation (Part 1) is worth **2** points.
- The improved version (Part 2) is worth **3** points.
- The report, combined with the code quality assessment, is worth **5** points.

On Themis, points are awarded per test case passed. For manual grading, we will select the *best* submission (i.e. the submission that was awarded the most points; if there are multiple, we select the most recent one).

Ideas for extensions (no bonus points)

- Write a function to print the trie. This may also be useful for debugging.
- Extend your program with command line arguments to read the dictionary and the text to be checked from two separate text files: `a.out -dictionary english.txt -text mytext.txt`.
- Store the trie in a file so that it can be reused quickly. (This can be a lot of work.)
- Look up and learn how to output colored text in the terminal. Instead of only printing unknown words, output the whole text and highlight the unknown words.